Question # 1:
(Tests Cases Included)

You are allowed to use built-in queues. The following code can be used for built-in queues

```
#include <queue>

queue<int> Q1;

Int first = Q1.front();   // for front element
Q1.pop();    // for dequeue operation
Q1.push(temp);  //for enqueue operation
```

Question # 2:
(Test Cases Included)

You need to make a node class and append a function that will insert the value into a linked list. Note that the ***find_mid_point*** will be outside of the LinkedList class

```
class Node {
public:
    T data;
    Node* next;

    Node(T value) : data(value), next(nullptr) {}
};

template <typename T>
class LinkedList {
public:
    Node<T>* head;

    LinkedList() : head(nullptr) {}

    void append(T value) {
}
};
```

Question # 3:
(No Test Cases)

Please note that this is just a hint to solving the problem, you are not restricted to using these functions

```
const int MAX_TREES = 100; // Maximum number of trees, adjust as needed

// node structure
struct node
{
    int key;
    struct node* left, * right;
};

// A utility function to create a new BST node
struct node* newNode(int item)
{
    struct node* temp = new node;
    temp->key = item;
    temp->left = temp->right = NULL;
    return temp;
}

// A utility function to do preorder traversal of BST
void preorder(struct node* root)
{
    if (root != NULL)
    {
        cout << root->key << " ";
        preorder(root->left);
        preorder(root->right);
    }
}

// function for constructing trees
void constructTrees(int start, int end, struct node* result[], int& count)
{
}
```

```
// Driver Program to test above functions
int main()
{
    node * totalTreesFrom1toN[MAX_TREES];
    int count = 0;

    // Construct all possible BSTs
    constructTrees(1, 3, totalTreesFrom1toN, count);

    /* Printing preorder traversal of all constructed BSTs */
    cout << "Preorder traversals of all constructed BSTs are \n";
    for (int i = 0; i < count; i++)
    {
        preorder(totalTreesFrom1toN[i]);
        cout << endl;
    }

    return 0;
}
```

Input : N = 3

Output should be preorder traversals:

1 2 3
1 3 2
2 1 3
3 1 2
3 2 1